2
Essential JavaScript for D3

This chapter reviews standard JavaScript technologies required by D3: data structures, modules, the **document object model** (**DOM**), and the Canvas API. It also includes a section about standard text formats used to distribute data, such as JSON and CSV. It is intended as a general reference to these topics. You can read the chapter in any order and skip the topics you are familiar with.

Besides the data structures used to store data in JavaScript, this chapter also reviews the main ECMAScript 2019 features used in modern JavaScript programs, which may be unfamiliar to some readers, such as arrow functions, promises, modules, async functions, and spread and rest operators.

A short reference to the standard DOM API is also included. It covers the basics, since D3 provides most of the tools you need to manipulate HTML or **scalable vector graphics** (**SVG**), but general knowledge about the DOM is important, for example, when integrating with Canvas, handling events, obtaining positions and dimensions, dealing with screen coordinate systems, or using other external resources.

Although primarily designed as a framework to generate SVG, D3 also supports Canvas rendering, a choice often motivated by performance optimizations. This chapter includes a short reference to the main methods available for the Canvas 2D context.

Finally, there is a brief overview of the main data formats used to store data in web-based visualizations: JSON, CSV, and XML. If you are comfortable with the concepts explored in this chapter, you can safely skip it entirely and refer to just the topics you need.

This chapter covers the following main topics:

- JavaScript data structures, methods and tools used for data manipulation
- JavaScript modules and asynchronous programming features
- The Document Object Model (DOM)
- The Canvas API
- Standard data formats

Technical requirements

All the code examples in this book use ES2019 (ES10) JavaScript. You should have a working knowledge of at least ES5 JavaScript to be able to follow the code examples. All the code snippets and examples used in this chapter can be found in the `Chapter02/` folder.

Essential web technologies for D3

You must know basic HTML, CSS, and JavaScript to start using D3. The more knowledge you have of these topics, the more you will benefit from D3, but you can always start with the basics and learn as you go.

For HTML and CSS, you should know how to:

- create a simple page, understand its hierarchical structure, and identify its main tags,
- use images, links, block and inline sections (`<div>`, `<span>`), lists, tables, and forms,
- declare CSS styles with simple selectors, basic properties, values, and units, in a `<style>` block or external style sheet imported with `<link>`,
- apply style rules to HTML elements via classes, IDs, and contextual selectors.

D3 is a JavaScript library, so you should know basic JavaScript, which means knowing at least how to:

- declare constants and variables,
- perform arithmetic, Boolean, and attribution operations,
- use loops and conditional expressions,
- declare, call, and use functions, with or without arguments, and use their return value,
- create strings and arrays, declare objects, and access their properties and methods.

This book uses modern JavaScript, so you should also be familiar with features such as `const/let`, arrow functions, promises, `await/async`, de-structuring, spread and rest operators, generators, iterators, maps, sets, modules (ESM), and `import/export`.

Does this seem like a lot? Don't worry! You can always fill in the gaps as you learn. If you know some HTML and CSS, and about the preceding basic JavaScript topics, you can start coding and use this chapter for a general overview of JavaScript data structures that are important in D3.

You should also read this chapter if you know JavaScript but aren't familiar with its modern features, since this book uses ES 2019 in all code examples.

Otherwise, you can safely skip this chapter.

How to explore the code in this chapter

The best way to learn a language is by writing, running, and debugging code. To run local files, you need a web server to serve JavaScript-powered HTML files. Using a code editor or **integrated development environment** (**IDE**) such as *WebStorm* or *Visual Studio Code* is best, since you can preview a web page with a click. These tools can also make it easier to manage your files: you can open the chapter's folder or the entire book repository as a project and quickly load and edit any file.

To run the snippets and short examples in this chapter, you don't need an IDE. An online JavaScript sandbox such as *CodePen* and *JSFiddle* is fine to run most examples. You can also try out simple commands by typing them in your browser's JavaScript console prompt for instant feedback.

The console is probably the most important learning tool, since it displays error messages caused by your code. It's very common to get a blank page when you expected something else and not have a clue why your code doesn't work. It may be a comma you forgot, or the internet is down, and some file was not loaded. If you have the console window open while you run your page, it will instantly tell you what's going on and give you relevant information (*Figure 2.1*).

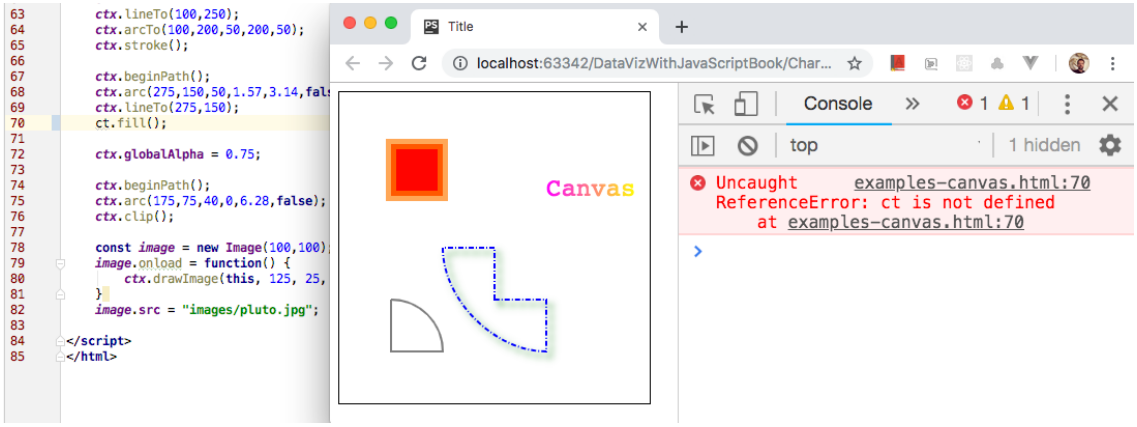

Figure 2.1 – Debugging with the JavaScript console

You can open the developer or browser tools as a frame or as a separate window in your browser. They are supported by all major browsers. Here are the menu paths for the JavaScript console in the latest versions (2024) of the three most popular browsers:

- **Chrome**: **View | Developer | JavaScript Console**
- **Firefox**: **Tools | Browser Tools | Browser Console**
- **Safari**: **Develop | Show JavaScript Console** (must first be configured in *Settings*)

The console's context is the currently loaded page, which gives access to functions of any library loaded with the `<script>` tag, and global variables declared in `<script>` blocks in the page. If they are in a module, access is more restricted, but they are still accessible via the debugger.

Modern JavaScript

Client-side applications, such as interactive web graphics, depend on browser support. This book assumes that your audience uses browsers that support *modern* JavaScript, usually meaning they support the standard *ECMAScript 2015* spec (also called ES6). However, all popular browsers in 2025 support at least *ECMAScript 2019* (ES10), which introduces a few more interesting features.

All examples in this book adopt the ES 2019 syntax and common practices, which include using:

- `const` and `let` instead of `var` to declare variables and constants,
- arrow functions (`d => d` instead of `function(d) { return d; }`) where appropriate,
- de-structuring, rest parameters, and spread operators `[...iterable]`,
- transformation pipelines by *chaining* methods such as `map()`, `reduce()`, and `filter()`,
- template string literals, defined using backticks, and multiline strings,
- generator functions and iterable collections, such as maps and sets,
- promises and `async/await` instead of nested callbacks for async operations, such as file loading,
- and standard modules (ESM).

Examples demonstrating all these features can be found in the `Chapter02/JavaScript/` folder. You can run them and view the results in your browser's console, or type in the code snippets directly at the console prompt as you read the next sections.

JavaScript data structures

Raw data used for visualizations is usually distributed in some kind of structure, usually a *list* or a *table*, stored in a standard format, such as CSV or JSON. To use the data, you need to fetch, parse, and store it locally as an *array* or *object* so it can be used in your chart.

Once your data is stored in a JavaScript data structure, it can be transformed by applying operations to its values. It's useful to have a good knowledge of the main structures, namely arrays, objects, functions, strings, maps, and sets, since your data will probably be in one of these formats. The upcoming sections briefly describe each one, with useful functions, methods, and operators used to manipulate the data. All code snippets are from scripts you can run from the `JavaScript/` folder.

Arrays

The main data structure you will use to store one-dimensional data is the JavaScript **array**. An array of values is all you need to make a simple bar or line chart. With an array of arrays, you can store two-dimensional data and create a scatterplot.

Arrays can be created by declaring a list of items within brackets, or simply a pair of opening-closing brackets to start with an empty array. Assignments store a reference to the array, like so:

```
const colors = ["red", "blue", "green"];
const geocoords = [27.2345, 34.9937];
const numbers = [1,2,3,4,5,6];
const empty = [];           // or new Array()
```

Items can then be accessed using their *index*, which starts counting from zero, as follows:

```
const blue = colors[1];
const latitude = geocoords[0];
```

In modern JavaScript, you can also extract array elements using a *de-structuring* assignment:

```
const [x, y, ...rest] = numbers;
console.log(x,y);   // 1,2
console.log(rest);  // [3,4,5,6]
```

De-structuring can also be used to retrieve arbitrary elements from the array:

```
const [,n,,,m] = numbers;
console.log(n);     // 2
console.log(m);     // 5
```

You can use the *spread* operator to create new arrays by concatenation:

```
const cats = ["meow", "garfield"];
const dogs = ["snoopy", "scooby"];
const pets = [...cats, ...dogs];      // ["meow","garfield","snoopy","scooby"]
```

An array has a `length` property that returns the number of elements it contains. This property is useful to loop using the array's index, since the index counts from 0 to `array.length-1`:

```
for(let index = 0; index < colors.length; index++) {
    console.log(colors[index]);
}
```

You can also loop over the elements of an array using the `of` operator, if you don't need the index:

```
for(let color of colors) {
    console.log(color);
}
```

Or, you can use the `forEach()` method, which runs a callback function for each element, giving access to the current element, its index, and the array itself inside that function:

```
colors.forEach((element, index, array) => {
    console.log(`color: ${element}, index: ${index}, colors[index]: ${array[index]}`);
});
```

This code may look strange if you're not familiar with ES 2015. It can also be written like this:

```
colors.forEach(
   function(element, index, array) {
       console.log("color: "+element+", index: "+index+",
                   array[index]: "+array[index]);
   }
);
```

Multi-dimensional arrays are created in JavaScript as arrays of arrays:

```
const points = [[200,300], [150,100], [100,300]];
```

You can retrieve individual items, like this:

```
const firstPoint = points[0];   // [200,300]
const middleX = points[1][0];   // 150
```

Or, use de-structuring assignments, like so:

```
const [firstPoint, [middleX,]] = points;
```

Nested loops can be used to loop through all the elements of a two-dimensional array:

```
for(row of points) {
    console.log(`row: ${row}`);
    for (data of row) {
        console.log(`   data: ${data}`);
    }
}
```

The following example does the same thing using a common `for` loop:

```
for(let i = 0; i < points.length; i++) {
    console.log(`row: ${points[i]}`);
    for (let j = 0; j < points[i].length; j++) {
        console.log(`   data: ${points[i][j]}`);
    }
}
```

Assigning an array to a new variable or constant doesn't make a copy of the array. It only copies its *reference* (the array's address in memory). The new binding will always *refer* to the original array:

```
const ncopy = numbers;
ncopy[2] = 8;       //  Now both ncopy AND numbers contain [1,2,8,4,5,6]
```

This can be useful if you pass an array to a function that will modify it:

```
function swap(narray) {
    narray.reverse();
}
swap(numbers);       // numbers is now [6,5,4,3,2,1]
```

But it can also lead to leaky code that is hard to debug. This can be avoided by returning a shallow *copy* of the array or only using methods that don't modify it. You can do this with the `spread` operator:

```
const temp = [...numbers];
temp.reverse();       // temp is reversed, numbers (other array) doesn't change
```

If the array is an array of objects or a multidimensional array, only the *references* are copied, not the items themselves. Consider the following example, where we clone the two-dimensional array `points`:

```
const aclone = [...points];   // points = [[200,300], [150,100], [100,300]]
```

Pushing new elements into the array or replacing top-level elements just affects the cloned array:

```
aclone.push([300,300]);        // aclone = [[200,300], [150,100], [100,300], [300,300]]
```

But if you make any changes in the contained items (which are not copies), the changes will be noted in *both* arrays, since they both contain references to the same objects:

```
aclone[0][0] = 0;              // now both points[0] and aclone[0] contain [0,300]
```

Now let's see some methods that can be used to modify an array.

Methods that modify an array

JavaScript provides many ways to extract and insert data into an array. It's usually recommended to use methods whenever possible. The methods in *Table 2.1* modify the array they are called from. The examples use the `colors` and `numbers` arrays as declared above.

Method	Description	Example
`push(`*item*`)`	Modifies the array, adding an item to the end. Returns the new size.	`colors.push("yellow");` `// ["red", "blue", "green", "yellow"];`
`pop()`	Modifies the array, removing and returning the last item.	`const green = colors.pop();` `// ["red", "blue"]`
`unshift(`*item*`)`	Modifies the array, inserting *item* at the beginning. Returns size.	`colors.unshift("yellow");` `// ["yellow", "red", "blue", "green"]`
`shift()`	Modifies the array, removing and returning the first item.	`const red = colors.shift();` `// ["blue", "green"]`
`reverse()`	Modifies the array, reversing it.	`numbers.reverse(); // [6,5,4,3,2,1]`

`sort()`
`sort(` *f(a,b)* `)`
	Modifies the array, sorting by string order or by a comparator function *f* provided as an argument. Returns the array itself.	`colors.sort();`
		`// ["blue", "green", "red"]`
		`numbers.sort((a,b) => b - a);`
		`// numbers = [6,5,4,3,2,1]`

`splice(`*p*`,`*r*`,...`*i*`)`	Modifies the array by removing *r* items from position *p*, and/or inserting new items in their place. Returns the removed items.	`const deleted = colors.splice(1);`
		`// colors = ["red"]`
		`// deleted = ["blue","green"]`

Table 2.1 – JavaScript array functions that modify the array. Code: JavaScript/2-Arrays-methods-1.html

The `splice()` method is perhaps the most complicated one and requires some additional examples. Starting with the four-element array:

```
const colors = ["red", "blue", "green", "yellow"];
```

The following code will remove two items from the middle. These items are returned as follows:

```
const deleted = colors.splice(1,2); // colors: ["red", "yellow"]
                                    // deleted: ["blue", "green"]
```

You can also use `splice()` to insert a new item in the array, like this:

```
const deleted = colors.splice(1,0,"pink");
// colors: ["red", "pink", "blue", "green", "yellow"]    // deleted: []
```

Or replace it, like this:

```
const deleted = colors.splice(1,1,"pink");
// colors: ["red", "pink", "green", "yellow"]       // deleted: ["blue"]
```

Or make several extractions, replacements, and insertions at the same time, like this:

```
const deleted = colors.splice(1,2,"a","b","c");
// colors: ["red", "a", "b", "c", "yellow"]         // deleted: ["blue", "green"]
```

The `sort()` method can receive a comparator function, which takes two arguments with elements to be compared. It uses these values to compute and return a positive, negative, or zero result.

Methods that don't modify the array

Several methods operate on the array, but don't modify it. Some search the array or compute values from its contents. Others transform the array and return a modified copy, leaving the original unmodified. *Table 2.2* demonstrates these methods with a simple example applied to the arrays declared above.

Method	Description	Example
`indexOf(`*item*`)`	Returns the index of the first occurrence of *item* in the array.	`const n = numbers.indexOf(3); // 4`
`lastIndexOf(`*item*`)`	Returns the index of the last occurrence of *item* in the array.	`const n = colors.lastIndexOf("blue"); // 1`
`includes(`*item*`)`	Returns *true* if an array contains *item*.	`const n = numbers.includes(3);  // true`
`find(` *f(e,i,a)* `)`	Returns the first element that satisfies the test function *f*.	`const e = numbers.find(n => n < 0);  // 2`
`concat(`*array*`)`	Returns a new array merging the current array with another.	`const eight = numbers.concat([7,8]);` `// [1,2,3,4,5,6,7,8]`

Method	Description	Example

`join()`
`join(`*delim*`)`
	Returns a comma-separated string of the elements in the array (the comma can be replaced by another delimiter).	`const csv = numbers.join();`
		`// "1,2,3,4,5,6"`
		`const conc = numbers.join(""); // "123456"`

`slice(`*begin*`,`*end*`)`	Returns a shallow copy of the array between *begin* and *end*.	`const mid = numbers.slice(2,4) // [3,4]`
`filter(`*f(e,i,a)*`)`	Returns a new array with the elements that pass the test implemented by *f*.	`const even =` `  numbers.filter(n => n%2==0); // [2,4,6]`
`map(`*f(e,i,a)*`)`	Returns a new array where each element is modified by the function *f*.	`const squares = numbers.map(n => n*n);` `// [1,4,9,16,25,36]`
`reduce(`*f(r,e,i,a)*`)`	Returns the result of an accumulation.	`const sum =` `  numbers.reduce((a,n) => a+n); // 21`

`forEach(`*f(e,i,a)*`)`	Executes the provided function *f* once for each array element.	`const squares = [];`
		`numbers.forEach(n => squares.push(n*n)`
		`// squares = [1,4,9,16,25,36]`

`flat(`*depth*`)`
`flatMap(`*depth*`)`
	Added in ES2019. Flattens the array up to *depth* levels. Using `flat()` is the same as using `flat(1)`.	`const a = [1,2,[3,4,[5,6],7],8];`
		`f1 = a.flat();  // [1,2,3,4,[5,6],7,8]`
		`f2 = a.flat(2); // [1,2,3,4,5,6,7,8]`

Table 2.2 – JavaScript array functions that don't modify the array. Code: JavaScript/3-Arrays-methods-2.html

Most methods that receive functions require at least one argument, which is the current element (*e*) of the array. The other arguments are the array's index (*i*) and the array itself (*a*). The `reduce()` function requires an additional argument *before* these, which is the accumulated result (*r*).

Objects

An **object** is an unordered data collection stored as key-value pairs. In JavaScript, objects can be created using constructors with the `new` keyword, or via prototypes, classes, or special functions. The simplest way is by declaring a comma-separated list of `property:value` pairs within curly braces, or just using empty curly braces to start with an empty object:

```
const color = {name: "red", code: "#ff0000"};
const empty = {};
```

Objects can include other objects, arrays, and functions. Functions within objects act as *methods* and access local properties. The following code declares two objects. The `city` object contains another object in its `location` property. The `circle` object contains a method called `area()`:

```
const city = {name: "Sao Paulo", location: {latitude: 23.555, longitude: 46.63},
              airports: ["SAO","CGH","GRU","VCP"]};
const circle = {
    x: 200, y: 100, r: 50,
    area: function() {
        return this.r * this.r * 3.14;
    }
}
```

The properties of an object can be accessed using the *dot* operator, or via a string containing the property's name in brackets. You can also call its methods using the dot operator:

```
const latitude = city.location.latitude;
const code = color["code"];
circle.r = 100;
const area = circle.area();
```

De-structuring can also be used to extract object properties:

```
const {x: v1, y: v2} = circle;
console.log(v1, v2);    // 200, 100
```

Iterating over all the properties of an object is possible using a `for-in` loop:

```
for(let key in circle) {
    console.log(key + ": " + circle[key]);
}
```

Objects can also be created with a **constructor**. JavaScript has several built-in constructors to create specific objects with the `new` keyword. For example, you can create an object that contains the current date/time using the built-in `Date` constructor as follows:

```
const now = new Date();
```

In modern JavaScript, you can create an object with the `Object.assign()` function, which magically creates properties from their binding names:

```
const x = 35.5, y = 122.9, z = 67.4;
const coords = Object.assign({}, {x}, {y}, {z});   // {z:67.4, x:35.5, y:122.9}
```

You can also use `Object.assign()` to merge objects:

```
const pos = Object.assign({name:"home"}, coords); // {name:'home',z:67.4,x:35.5,y:122.9}
```

And make shallow copies of objects (also possible with the spread operator):

```
const city2 = Object.assign({}, city);
const city3 = {...city};            // using the spread operator
```

A typical dataset used by a simple chart usually consists of an array of objects:

```
var continents = [
    { name: "Asia", areakm2: 43820000 },
    { name: "Europe", areakm2: 10180000 },
    { name: "Africa", areakm2: 30370000 },
    { name: "South America", areakm2: 17840000 },
    { name: "Oceania", areakm2: 9008500 },
    { name: "North America", areakm2: 24490000 }
];
```

Data can be extracted using the *dot* operator or with de-structuring assignments:

```
const oceania = continents[4].name;
const [,,{areakm2:ofAfrica}] = continents;     // ofAfrica = value of areakm2 of [2]
console.log(`Area of Africa = ${ofAfrica}`);   // 30370000
```

Like arrays, an object binding contains a *reference* to the object. If you make a shallow copy of an object array, as shown here, you simply copy its references like this:

```
const aclone = [...continents];
```

Removing, inserting, or replacing objects will only affect the copy:

```
const asia   = aclone.shift();      // removes first item - affects only aclone
```

But if you access any object and change its properties, this will be reflected in both the original array and its copy. The following code changes the data for the first object in `aclone` and `continents`:

```
aclone[0].name = "Eurasia";
aclone[0].areakm2 = aclone[0].areakm2 + asia.areakm2;
```

Properties and functions can be added to objects. It's common to write code that declares an empty object in a global context so that operations in other contexts add data to it:

```
const map = {};
function setCoords(c) {
    map.latitude = c.x;   // adding a latitude property
    map.longitude = c.y;  // adding a longitude property
} // after setCoords(coords), map will be {latitude:35.5, longitude:122.9}
```

JSON is a data format based on JavaScript objects, which is stored as a string. It has the same structure as a JavaScript object but requires that property keys be placed within double quotes:

```
const json = '{"name": "Sao Paulo",
               "location": {"latitude": 23.555, "longitude": 46.63},
               "airports": ["SAO","CGH","GRU","VCP"]}';
```

To use a JSON string in JavaScript, you must parse it. You can do that with the `JSON.parse()` function. Read more about JSON in the *Data formats* section.

Maps and sets

Besides arrays, ES2015 introduced two new data structures: `Map`, an associative array with key-value pairs, and `Set`, which doesn't repeat values. Both can be transformed to and from arrays.

You can create a new `Set` object using the `Set()` constructor:

```
const set = new Set();
```

And add elements to it using the `add()` method:

```
set.add(5);
set.add(7);
```

If you try to add elements that already exist, they won't be included in the set:

```
set.add(1);
set.add(5);             // not added
set.add(7);             // not added
console.log(set.size);  // the size property returns 3
```

You can check if a set contains an element with `has()`:

```
console.log(set.has(3)); // false
```

And convert a `Set` object into an array using the spread operator (or `Array.from()`):

```
const array1 = [...set]; // spread operator
```

A `Map` can be more efficient than using an object to store key-value pairs in an associative array:

```
const map = new Map();
map.set("a", 123)
map.set("b", 456);
```

You can then retrieve the *value* for each *key* using `get()`:

```
console.log(map.get("b"))
```

Or, using iterative operations and the `keys()`, `values()`, and `entries()` methods:

```
for(let key of map.keys()) {
    console.log(key, map.get(key))
}

for(let [key, value] of map.entries()) {
    console.log(key, value)
}

map.forEach((key, value) => console.log(key, value));
```

Maps can also be converted into arrays with the spread operator or using `Array.from()`:

```
const array2 = [... map.values()];     // an array of values
const array3 = Array.from(map.keys()); // an array of keys
```

Although objects can be used for maps and sets, it's much simpler and recommended to use `Map` and `Set`.

Strings

String literals can be created with characters between quotes. There is no difference between single or double quotes. It's just a matter of style.

Inside strings, special characters such as newlines, tab characters, and Unicode character codes must be preceded by a backslash. To use a backslash in a string, you must escape it with another backslash.

Backslashes can also be used to print quotation marks, apostrophes, and backticks, if necessary:

```
const s = "This is a backslash \\ and this is a double quote \"";
```

It's usually easier, and recommended, to avoid backslashes when possible, since they make strings harder to read. The following string, for example, uses single quotes to avoid having to escape the double quotes:

```
const json = '{"day":14,"month":4,"year":2023}';
```

Strings can also be created by concatenating strings and other types, which are automatically converted to strings before the concatenation.

```
const result = 'The square root of ' + x + ' is ' + Math.sqrt(x);
```

Any + inside a string expression is always a concatenation. To perform an addition before concatenating, use parentheses; otherwise, the numbers will be concatenated:

```
const result2 = 'The sum of '+a+' and '+b+' is '+ (a+b);
```

If you have a number stored in a binding of type *String* (typical for data in forms and attributes), it needs to be converted into type *Number* before using it in an arithmetic operation:

```
const v1 = "5";   // type String: if used in addition, concatenates: v1 + 10 = "510"
const v2 = +v1;   // converts to type Number: v2 + 10 = 15
```

This can be done with methods, such as `parseInt()` or `parseFloat()`, but it's much simpler using the + operator, as shown above.

```
const v3 = parseInt(v1);    // also converts to type Number
```

ES2015 introduced two new types of strings: **template literals** and **multiline strings**. Multiline strings can be created by adding a single backslash at the end of each line:

```
const line = "Multiline strings can be \
              created adding a backslash \
              at the end of each line";
```

But you don't need any backslashes if your string is quoted with *backticks*:

```
const line2 = `Multiline strings can also be
               created without any backslashes
               if you use backticks`;
```

Strings quoted with backticks also have another superpower. You can create template literals with them by including JavaScript expressions inside ${} placeholders. The string will be generated when the expression is executed. This is much better than concatenating with the + operator:

```
const template = `The square root of 2 is ${Math.sqrt(2)}`;
```

Compare this with the concatenation shown before in this section, which produces the same result.

Strings contain methods (listed in *Table 2.3*) that can be used to check and match substrings, locate a certain character or substring, convert the string into an array, and replace characters or substrings. They all return new strings or other types. No methods modify the original strings.

Method	Description	Example
`startsWith(`*s*`)`	Returns *true* if the string starts with the string passed as a parameter.	`const s = "This is a test string"` `const r = s.startsWith("This"); // true`
`endsWith(`*s*`)`	Returns *true* if the string ends with the string passed as a parameter.	`const s = "This is a test string"` `const r = s.endsWith("This"); // false`
`indexOf(`*s*`)`	Returns the index of the first occurrence of a substring.	`const k = "Aardvark"` `const i = k.indexOf("ar"); // i = 1`
`lastIndexOf(`*s*`)`	Returns the index of the last occurrence of a substring.	`const k = "Aardvark"` `const i = k.lastIndexOf("ar"); // i = 5`
`charAt(`*i*`)`	Returns char at index *i*. Also supported as '*string*'[*i*]	`const k = "Aardvark"` `const result = k.charAt(4);  // "v"`

Method	Description	Example

`substring(`*s*`,`*e*`)`	Returns a substring between *start* (included) and *end* indexes (not included).	`const k = "Aardvark"`
		`const result = k.substring(1,6);`
		`// "ardva"`

`trim()`	Removes whitespace from both ends of a string.	`const text = "  data  "` `const r = text.trim();  // r = "data"`

`split(`*delim*`)`
`split(`*regx*`)`
	Splits a string by a delimiter string or regular expression and returns an array.	`const s = "This is a test string";`
		`const result = s.split(/\s/);`
		`// ["This","is","a","test","string"]`

`match(`*regx*`)`	Returns an array as the result of matching a regular expression against the string.	`const k = "Aardvark"`
		`const v = k.match(/[a-f]/g);`
		`// v = ["a", "d", "a"]`

`replace(`*regx*`,`*r*`)`
`replace(`*s*`,`*t*`)`
	Returns a new string replacing the match of *regx* applied to the string with replacement, or all occurrences of the source string with a target string.	`const k = "Aardvark"`
		`const a = k.replace(/a/gi, 'u')`
		`// a = "uurdvurk"`
		`const b = k.replace('ardv', 'ntib')`
		`// b = "Antibark"`

Table 2.3 – Methods that can be called from strings. Code: JavaScript/7-Strings-methods.html

Some methods receive regular expressions as a parameter. Regular expressions are written without any quotes and between slashes, with optional flags (e.g., *global* `g` and/or *ignore case* `i`).

The `split()` method is the opposite of the `join()` method used in arrays. You can use it to transform a string into an array and then use `join()` to convert it back into a string.

See these and other examples in `JavaScript/7-Strings-methods.html`.

Method chaining

Methods that return the array itself or a modified copy of the array can be chained together with other methods in a transformation pipeline. Each of the methods in the following code operates on the arrays that are returned in the previous step. The last method returns the result:

```
const sum2 = numbers.slice(2,4)                    // Array: [3,4]
                    .concat([7,8,9,10])            // Array: [3,4,7,8,9,10]
                    .filter(n => n % 2 !== 0)      // Array: [3,7,9]
                    .map(n => n*n)                 // Array: [9,49,81]
                    .reduce((acum, n) => acum + n) // Number: 139
```

Methods that modify the array, such as `sort()` and `reverse()`, can also be chained, since they also return a reference to the modified array.

Note that since each step returns a different array, altering the order may also alter the results. After `reduce()`, you can no longer call *Array* methods, but you can call *Number* methods, such as `toString()`, and then, since `toString()` returns a *String*, you can call other *String* methods.

In the following example, a transformation pipeline is used to convert the string "2023-04-12" into the JSON string `{"day":12,"month":4,"year":2023}`:

```
const labels = ["year", "month", "day"];
const string = "2023-4-12";

const json = string.split("-")          // Now type is Array
                   .map( (dat,idx,arr) => arr[idx] = `"${labels[idx]}": ${+dat}`)
                   .reverse()
                   .join(", ")          // Now type is String
                   .replace(/^/g, "{ ")
                   .replace(/$/g, " }");
```

The JSON string can then be converted into a JavaScript object using:

```
const obj = JSON.parse(json);    // Object
```

This is not the best way to convert a string into an object, but it is a good and simple demonstration of how to chain methods (see better alternatives in `JavaScript/8-Method-chaining.html`).

Method chaining is great when you need to call several methods on an object, such as when applying properties and styles to an object selection, in D3.

Declaring functions and methods

Functions are created either using the `function` keyword or the arrow notation. Here are three equivalent ways to declare a function that receives two arguments and returns their sum:

```
function add(a,b) {
    return a+b;
}
const add = function(a,b) {
    return a+b;
}
const add = (a,b) => a+b;
```

The last one is called an *arrow function* and was introduced in ES2015. They are all called as follows:

```
const result = add(4,5);   // 9
```

Arrow functions are typically used for short operations, mostly one-liners that don't require a local `this` context. If there is only one argument, parentheses are optional. If the body contains a single expression, you don't need braces, and the value is automatically returned. The following two forms are equivalent:

```
const squareDiagonal1 = (side) => {
    return Math.sqrt(2) * side;
```

```
};

const squareDiagonal2 = side => Math.sqrt(2) * side;
```

A function can be declared in the scope of an object, behaving as the object's **method**. In the following snippet, the `rect` object has a method called `diagonal()`:

```
const rect = {w: 3, h: 4};

rect.diagonal = function() {
    return Math.sqrt( this.w*this.w + this.h*this.h );
}
```

You can call the method using a reference to the object that contains it:

```
const result = rect.diagonal();   // 5
```

Methods can access the object's properties using the `this` keyword, which is a reference to the object that owns the method. In a top-level function, `this` points to the global object (the `window` object, when the code runs in a browser). However, the `this` keyword only references the object that owns the method if the method is declared with the `function` keyword. In arrow functions, `this` always points to the global object, so the following won't work, as `h` and `w` will now be `undefined`:

```
rect.diagonal = () => Math.sqrt( this.w*this.w + this.h*this.h );   // NaN!
```

Functions that receive other functions as arguments are called **higher-order functions**. An example is the following method, which receives a function that takes the rectangle's width and height:

```
rect.operation = function(f) {
    return f(this.w, this.h);
}
```

You can pass any function that requires two arguments, such as the `add()` function declared above:

```
const v1 = rect.operation(add);   // 7
```

If you don't plan to reuse the function, you can declare it immediately as an anonymous function when you pass it as an argument.

```
const v2 = rect.operation( function(a,b) { return a*b; } );   // 12
```

To make the code easier to read, you should indent your code, as follows, especially if it contains more than one line of code:

```
const v2 = rect.operation(function(a,b) {
    return a*b;
});
```

Functions that are declared in arguments are usually short and sometimes can be written in one line. This is the most typical use case for arrow functions, which fits in one line:

```
const v2 = rect.operation( (a,b)  => a*b );
```

You can see a few more examples with functions in `JavaScript/9-Functions.html`.

Generators and iterators

ES2015 introduced a special type of function called a **generator**, that can be used to create iterators. To declare a generator, use the `function` keyword followed by a *. The `yield` keyword returns the result when `next()` is called from the iterator (`JavaScript/10-Generators.html`).

The following generator can be used to provide an iterator that produces different colors:

```
function* randomColor() {
    while (true) {
        const r = Math.floor(Math.random()*256);
        const g = Math.floor(Math.random()*256);
        const b = Math.floor(Math.random()*256);
        yield `rgb(${r},${g},${b})`;
    }
}
```

The `randomColor()` generator is called to produce an iterator, which is the following `color()` function:

```
const color = randomColor();
```

Now you can return a different color every time you use the iterator to call `next()`:

```
const c1 = color.next().value;
const c2 = color.next().value;
const c3 = color.next().value;
```

Generators can be used whenever you need to generate values in a non-continuous manner.

Large JavaScript apps usually distribute their code in separate files. In the next section, you will learn the best way to do this and how to efficiently manage asynchronous operations in modern JavaScript.

Modules and asynchronous JavaScript

Traditionally, JavaScript has used the `<script>` tag to load external files and libraries. While not a problem in small apps, this can be hard to manage when there are multiple scripts that depend on each other. It also makes it more challenging to write libraries, which require extra code to handle potential naming conflicts, order of execution, browser compatibility, and other problems.

Many of these issues can be reduced using a server-side development environment with bundling, dependency management, and code generation, but this solution requires a complex setup and is overkill for small projects. Later solutions, such as **universal module definition** (**UMD**), solved some of these problems in pure client-side JavaScript but also introduced a complex syntax.

In 2015, JavaScript finally proposed a standard module system called **ECMAScript modules** (**ESM**), which is safer, more efficient, and easier to use. Today, ESM is supported in all major browsers and is the recommended way to distribute code in modules.

Module loading is an asynchronous process, like many other processes in JavaScript, such as fetching files, querying databases, handling events, and animations. Before ES 2015, synchronization required complex callback structures. In modern JavaScript, you have *promises* and the `async/await` keywords, making it much easier to write asynchronous code. We will explore both these topics in this section.

Creating and importing modules

To use modules, you just need to learn how to use two new keywords: `import` and `export`.

Let's start with a simple page containing a `<script>` block (see `JavaScript/14-modules.html`). To use imports, it must be declared with the `type="module"` attribute. The following script imports and uses two functions from the `tableFunctions.js` file (located in `JavaScript/modules/`):

```
<script type="module">
    import {append, makeTable} from './modules/tableFunctions.js';

    const data = [ ... ];
    const table = makeTable(data);   // using imported functions
    append(table);
</script>
```

The `tableFunctions.js` file can be loaded as a module because it declares its exports. Note the `export` keyword before the declarations for `makeTable()` and `append()`, which allow them to be used outside the scope of the module. All other artifacts are local and inaccessible outside the module:

```
import style from './style.js';         // uses default export
import * as color from './colors.js';   // uses prefixed exports

style.innerText =
    ` table,td,th {border: solid 2px ${color.red};}
      td,th {padding: 5px;}
      th {border-color: ${color.random()};}
    `;
export function makeTable(data) { /* ... */ }   // these functions are exported
export function append(table) { /* ... */ }
function makeRow(rowData) { /* ... */ }     // these functions are not exported
function makeCell(cellData) { /* ... */ }
```

A module can import other modules, as shown in the first two lines of the preceding code: the `tableFunctions.js` module is importing two other modules from the same directory.

The `style.js` module provides an object as its *default* export, which is used to set some styles in `tableFunctions.js` by creating a `<style>` block in the page. This is how it is declared in `style.js`:

```
export default document.documentElement.appendChild(document.createElement('style'));
```

All the exported objects from the `colors.js` module were imported into `tableFunctions.js`, which uses them with a `color` prefix (the `color.red` constant and the `color.random()` function).

In `colors.js`, you can see that they were all exported. This time, instead of using the `export` keyword before each declaration, the exports were declared in a list:

```
const red = '#ff0000';
const blue = '#0000ff';
const yellow = '#ffff00';
function random() {
    return `#${Math.floor(Math.random() * 0xffffff).toString(16)}`;
}
export {red, blue, yellow, random};    // list of exported functions and constants
```

In most of the D3 examples used in this book, we import the entire D3 library as a module using the standard `d3` prefix for all methods. You can, of course, choose another prefix, or even import just the function you will use without any prefix. See `Chapter01/Imports/` for different ways you can import the D3 library and separate modules.

Modules make your code stricter: constants and variables within a module aren't global unless exported. Although this makes your code safer, it also complicates debugging with the console. You can, however, enter the module's scope via the debugger or create a bypass by copying references to the `window` object, if you really need total access. Most examples in this chapter are in `<script>` blocks and don't use modules. D3 examples used in the rest of the book *do* use them, but include bypasses when necessary.

Promises

A **promise** is an object returned by an asynchronous function, which may (or may not) contain the result of a usually time-consuming asynchronous operation. Calling the `then(`*function*`)` method on a promise calls the provided function with its result when the operation succeeds. Since the result of `then()` is also a promise, you can chain several `then()` methods together, in a sequence.

An example of a function that returns a promise is `fetch()`, which performs an HTTP request. It is used in the following example to fetch a file from a local URL received via an HTTP response. The code assumes the operation will not fail and, as soon as the file is loaded, it will print the text:

```
fetch('../data/PlanetDiameters.csv').then(response => console.log(response.text()));
```

But the file might not exist. To deal with this possibility, you can check the status before attempting to use the data, and handle the possible error using `catch()`:

```
fetch('../data/BadFile')
    .then(response => {
        if (response.ok) return response.text();
        else if (response.status == 404) throw new Error(`File not found!`);
    })
    .then(text => document.getElementById('status').innerHTML = "Success: " + text);
    .catch(error => document.getElementById('status').innerHTML = error);
```

If, after loading the text, you decide to process it and perform any other transformations before using it, you might decide to do this asynchronously, especially if these operations take some time. This can be achieved by chaining `then()` methods (see `JavaScript/11-Promises.html`):

```
fetch('../data/PlanetDiameters.csv')
    .then(response => response.text())     // HttpResponse object
    .then(text => text.substring(14,19))   // String object
    .then(word => document.getElementById('result').innerHTML = word);
```

To perform several asynchronous operations simultaneously and only start using the data when all of them are complete, you can use `Promise.all()`. The following example computes the total length of three files after they are all loaded (see `JavaScript/12-Promise-all.html`):

```
const filenames = ['continents.csv','continents.json','continents.xml'];
const promises = filenames.map(file => fetch(`../data/${file}`)
                          .then(resp => resp.text()));
Promise.all(promises)
   .then( files => {
       const total = files.map(file => file.length).reduce( (tot, len) => tot + len);
       console.log(`Total size (bytes): ${total}`);
   });
```

The preceding code creates an array of `promises` to fetch each file and return it as a string. The files are fetched in `Promise.all()`. The `then()` clause is only called when all responses are received.

Await and async functions

Preceding a promise with the `await` keyword blocks its execution until the promise is fulfilled or rejected. For example, instead of placing your code in the `then()` callback:

```
fetch('file').then(result => {
    /* ... do something with result …*/
}
```

You can keep the normal program flow, placing it after the asynchronous expression:

```
const result = await fetch('file');
/* ... do something with result … */
```

You can only use `await` inside a module (e.g., `<script type="module">` block) or asynchronous function. It can make your code easier to read, especially when handling multiple promises. Functions declared with `async` always return a promise. For example, let's place our `Promise.all()` code in a function:

```
async function fetchFiles(proms) {
    const files = await Promise.all(proms);
    return files.map(file => file.length).reduce( (tot, len) => tot + len);
}
```

Since `fetchFiles()` is asynchronous, it returns a promise, so it must be handled with `then()` or `await`:

```
const total = await fetchFiles(promises);
console.log(`Total size (bytes): ${total}`);
```

You will find the code from this section in `JavaScript/13-Async-await.html`.

This ends our brief tutorial on JavaScript. Many features were purposely left out of this introduction, but what was covered here is enough for you to code in D3. For more details, check the *References* section at the end of this chapter. More examples can also be found in the `JavaScript/` folder.

The DOM

An HTML or XML document is normally described with tags, but it can also be specified using JavaScript commands with the **DOM**: a language-neutral API that represents an HTML or XML document as a *tree*.

Consider the following HTML document (`DOM/1-Static-page.html`):

```
<html>
<body>
    <h1>Simple page</h1>
    <p>Simple paragraph</p>
    <div>
        <img src="pluto.jpg" width="100"/>
        <p>Click me!</p>
    </div>
</body>
</html>
```

This page builds a tree of interconnected *nodes* containing HTML elements, attributes, and text (*Figure 2.2*). Note that the generated tree includes the `<head>` element, which is not in the code.

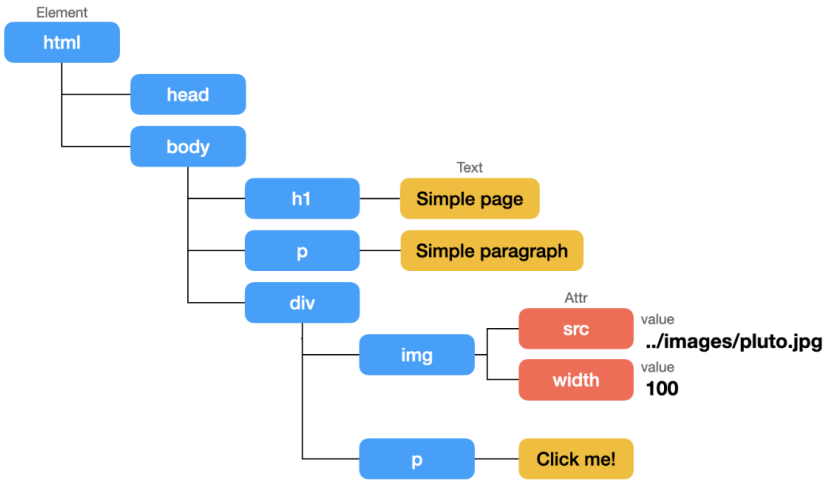

Figure 2.2 – DOM tree generated from HTML

The DOM API provides a JavaScript object called `document` that gives access to the page and allows you to use properties and methods to navigate through an existing tree or create a new one. Here's how you could use it to generate the preceding HTML (`DOM/2-Dynamic-page.html`):

```
const html = document.documentElement;
const body = html.appendChild(document.createElement("body"));
document.body = body;

const h1 = document.createElement("h1");
h1.innerText = "Simple page";
body.appendChild(h1);

const p = document.createElement("p");
p.innerText = "Simple paragraph";
body.appendChild(p);

const div = document.createElement("div");
div.innerHTML = "<img src='../images/pluto.jpg' width='100'><p>Click me!</p>";
body.appendChild(div);
```

Of course, it's still easier to write tags, but JavaScript gives you the power to make the structure and content *dynamic*. With DOM, you can add, move, or remove elements, change attributes and contents, navigate the tree, select or search for elements or data, bind styles, and attach event handlers to elements.

For example, add the following code to dynamically create a `<p>` containing `New line` at each user click:

```
div.addEventListener("click", function() {
    document.createElement("p").innerHTML = "New line";
    this.appendChild(p);
});
```

Normally, you don't write a full document using DOM, but only the dynamic parts, leaving the static parts in HTML. Special methods allow you to select static elements and obtain an object that you can use in code. A typical JavaScript-powered web page is shown here (`DOM/3-Page-and-script.html`):

```
<html><body>
  <h1>Simple page</h1><p>Simple paragraph</p>
  <div id="my-section">
    <img src="pluto.jpg" width="100"/><p>Click me!</p>
  </div>
  </body>
  <script>
    const div = document.getElementById("my-section");
    div.addEventListener("click", function() {
        const p = document.createElement("p");
        p.innerHTML = "New line";
        this.appendChild(p);
```

```
    });
  </script>
</html>
```

For data-driven documents, you can use DOM scripting to bind data stored in arrays and objects to an element's attributes, changing its dimensions, color, text contents, and position.

Inspecting generated code

In most browsers, right-clicking anywhere on the page and selecting **Inspect** opens a frame containing the generated code (*Figure 2.3*).

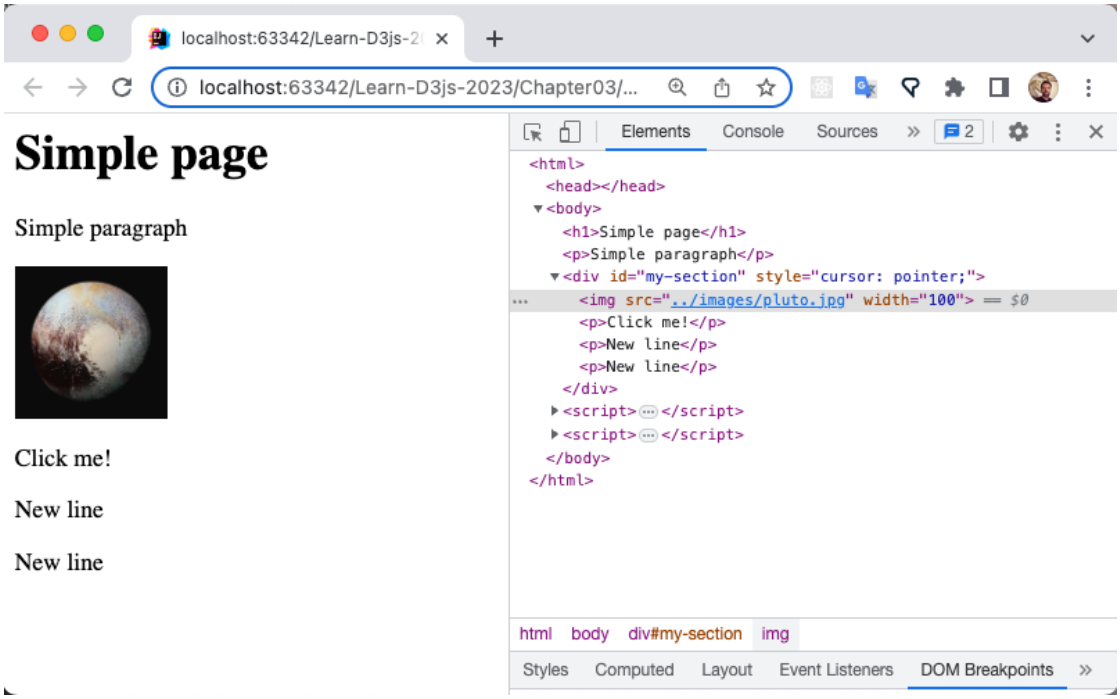

Figure 2.3 – Using the element inspector to view HTML elements generated with the DOM API

With this tool, you can watch DOM code generation in real time. Click on the **New line** text, and when a new paragraph appears on the screen, a new `<p>New line</p>` node is immediately appended to the generated code tree. It's a great resource for debugging DOM applications.

Interface hierarchy

The DOM is a huge API containing objects, methods, and properties, described by interfaces implemented by browsers and other tools, which form a hierarchy (*Figure 2.4*). Every node in a page implements a specific interface. Some interfaces are shared by all DOM implementations, but a specific application can add its own set. For example, some interfaces only apply to HTML, others work only in SVG, and so on.

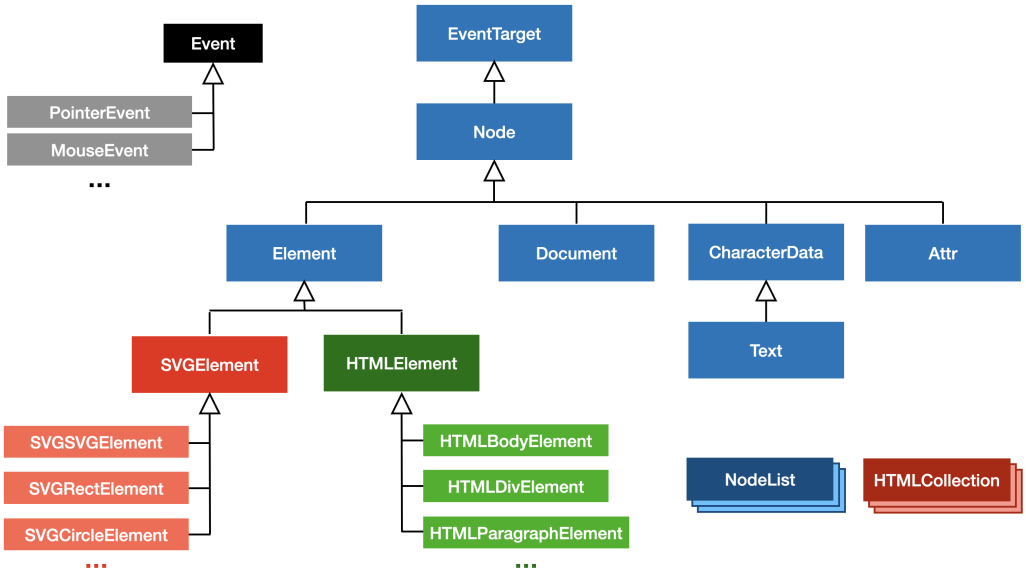

Figure 2.4 – A simplified hierarchy showing the main DOM interfaces used in HTML pages with embedded SVG

Methods in parent interfaces are inherited by their children, which may add more methods. Knowing a node's type and interface is important to know which methods you can call. For example, the `document` object (the HTML page) implements `HTMLDocument`. From a `document` object, you can invoke any method defined in `HTMLDocument`, `Document`, `Node`, or `EventTarget`.

A node's type can be discovered by comparing its `nodeType` property to a constant defined in `Node`:

```
if (someNode.nodeType === Node.ELEMENT_NODE) {
    // someNode is an Element (any <svg> or <html> element)
}
```

To test whether an interface is implemented by a node, use `instanceof` before attempting to call its methods or access its properties:

```
if(someNode instanceof SVGCircleElement) {
    // someNode is an SVG <circle> (it has r, cx and cy properties)
}
```

To know the exact interface implemented by a node, you can use the `constructor.name` property:

```
console.log( document.createElement("div").constructor.name ); // HTMLDivElement
```

The DOM API is very large, but knowing just a few of its methods and properties is all you need. The next sections will provide a good overview.

Retrieving elements

The document node is a special node that represents the entire page in HTML and contains a single child, which is the <html> element node. Its methods are defined in the HTMLDocument interface, which inherits from the Document interface. The methods listed in *Table 2.4* are used to retrieve elements declared (with HTML or SVG tags) in a page and may return one or more elements.

Method or property	Interface	Description
getElementById(*id*)	Document	Returns a single Element identified by *id*. Returns *undefined* if it is not found.
querySelector(*selector*)	Document	Returns the first Element object that matches the provided *selector* (see *W3C selectors*).
querySelectorAll(*selector*)	Document	Returns a NodeList (array) of Element objects that match the provided selector (see *W3C selectors*).
getElementsByTagName(*tag*)	Document	Returns a NodeList (array) containing all elements that match the *tag* name.
getElementsByClassName(*class*)	HTMLDocument	Returns an HTMLCollection (use Array.from() to convert to an array) containing all the elements that match the *class* name passed as a parameter.
documentElement	Document	References the element at the root of the document (e.g., <html> in web pages).

Table 2.4 – Main methods and properties used to retrieve elements from a document

Note that multiple elements sometimes use different interfaces. The NodeList interface is implemented as an array in JavaScript, so you can simply append an array index after the method call to retrieve a specific element. For example, to obtain the element for the <body> tag:

```
const body = document.getElementsByTagName("body")[0];   // the first <body> Element
```

However, the HTML-specific HTMLCollection is not an array. You must pass the index to its item() method to get a specific element. But you can also convert it to an array before using it:

```
const collection = document.getElementsByClassName("chart");
const element = Array.from(collection)[0];
```

You can also obtain the element using collection.item().

Creating nodes

Table 2.5 lists methods from the `Document` interface used to create new (detached) elements:

Method	Interface	Description
`createElement(`*tag*`)`	`Document`	Creates a detached `Element` node and returns its reference.
`createElementNS(`*namespace*`,` *tag*`)`	`Document`	Creates a detached `Element` node in a provided *namespace* and returns its reference.
`createTextNode(`*text*`)`	`Document`	Creates a detached `Text` node and returns its reference.

Table 2.5 – Main methods used to create elements and text nodes

You need `createElementNS()` to create SVG elements in an HTML page. If you use `createElement()`, the tag will still be created, but it won't be treated as SVG and won't be rendered.

Modifying the tree

The `document` methods in *Table 2.5* create nodes and return a reference, but don't attach them to the tree. You need `appendChild()` for this. *Table 2.6* lists methods you can use to modify a tree.

Method	Interface	Description
`append(`...*node*`)`	`Element`	Attaches *nodes* to the current element, *after* any existing children.
`prepend(`...*node*`)`	`Element`	Prepends *nodes* to the current element, *before* any existing children.
`remove()`	`Element`	Removes the current element from the tree.
`appendChild(`*element*`)`	`Node`	Attaches the *element* as a child of the current node.
`removeChild(`*element*`)`	`Node`	Detaches the child *element* from the current node.

Table 2.6 – Main methods used to add and remove nodes to and from the DOM tree

The older `appendChild()` and `removeChild()` methods duplicate the functionality of `append()` and `remove()`, but are supported by all nodes. You need them to add or remove the `<html>` root node (the only child in the `document` node), since `Document` doesn't support `append()` and `remove()`.

Attributes, text, and styles

Table 2.7 lists methods and properties to access an element's attributes and text nodes, and apply styles:

Method or property	Interface	Description
`setAttribute(`*name*`,`*value*`)`	`Element`	Sets an attribute for this element with *name* and *value*.
`getAttribute(`*name*`)`	`Element`	Returns the value of a named attribute of this element.
`hasAttribute(`*name*`)`	`Element`	Returns *true* if the named attribute exists in this element.
`innerText`	`HTMLElement`	In HTML documents, this read-write property creates a text node and appends it to the element. Does *not* work in SVG.
`innerHTML`	`Document`	In HTML documents, this read-write property appends an HTML fragment as a child. In modern browsers, it also works in SVG.
`style`	`HTMLElement` `SVGElement`	In SVG or HTML documents, allows read-write access to the element's CSS styles.

Table 2.7 – Main methods and properties used to get/set attributes, text, and inline CSS styles

An `Element` can get and set attributes with the `get/setAttribute()` methods. You can also set, read, and update an element's styles through the `style` property, but the name must be converted to *camelCase* if it has a hyphen. For example, to set the `background-color`, use:

```
element.style.backgroundColor = 'blue';
```

Text nodes can be created with `document.createTextNode("string")`, but to get the text, you need to call `CharacterData` methods. In an HTML page, it's much simpler to use a property such as `innerHTML`.

Events

`Event` objects store data about user events, page loading, form submission, and so on. Since `EventTarget` is at the root of the DOM hierarchy, any node (`Element`, `Document`, `Text`, etc.) can be a target and call methods to dispatch events, add, or remove an event listener. *Table 2.8* lists the main methods and properties.

Method or property	Interface	Description
`addEventListener(`*type*`,`*func*`)`	`EventTarget`	Attaches an event handler to a node, receiving a *type* (e.g., `'click'`, `'key'`) and *function*, which receives an `Event` object.
`removeEventListener(`*type*`,`*func*`)`	`EventTarget`	Detaches a registered event listener *function*.
`preventDefault()`	`Event`	Prevents a default event from being triggered.

Method or property	Interface	Description
`stopPropagation()`	`Event`	In nested elements, stops events bubbling up the DOM tree.
`target`	`Event`	A reference to the node where the event object was created.

Table 2.8 – Main methods and properties used in event handling

Event handling was demonstrated in the dynamic page at the beginning of this section to create a new paragraph responding to a `'click'` event. See more examples in the `DOM/` folder.

W3C selectors

The *Selectors W3C Specification* is one of the standards used to select nodes in HTML and XML documents. CSS uses selector strings to apply style properties to elements. In DOM, they are used to reference elements via `document.querySelector()` and `document.querySelectorAll()`.

Most expressions in D3 use a single selector of one of the six types described in *Table 2.9*. Selections always select *elements* and are applied to a *context*, resulting in an empty set or a set of elements.

Type	Examples	Description
Type selector	`div` `rect`	Selects all elements with a matching tag name. Can be used as a prefix to attribute, class, ID, and pseudo-class selectors.
Universal selector	*	Selects all elements. Optional when used with other selectors.

Attribute selectors	`[height]` or `*[height]`
	`[height="300"]`
	`div[height=300]`
		Selects elements that contain the named attribute or name/value expressed as `[name="value"]`. Matches can also be specified using wildcards (~,|,^,$,*) for non-exact values. If preceded by * or nothing, applies to any element type; otherwise, restricts the scope to the prefixing selector. Quotes are optional.

Class selectors	`.bar`
	`.horiz`
	`.bar.horiz` or `.horiz.bar`
	`div.bar`
		Selects elements that are part of the named class, preceded by a dot (`.`) character. If the dot is preceded by * or nothing, applies to any element; otherwise, it restricts the scope to the prefixing selector. To match elements that belong to multiple classes, selectors for each class are concatenated by dots in any order.

ID selectors	`#main-table`	Selects a single element that is tagged with a named ID, preceded by a hash (#) character.

Type	Examples	Description

Pseudo-classes	`:enabled`
	`:first-child`
	`div:nth-child(2n+1)`
	`p:nth-of-type(2n)`
	`g:empty`
	`:not(`*selector*`)`
		Selects elements based on features such as state, properties, position, or negation. If preceded by * or nothing, applies to any element type; otherwise, it restricts the scope to the prefixing selector.

Table 2.9 – W3C selectors used in D3. All selectors select elements

Combinators select elements represented by the *last simple selector* in an expression, where the preceding simple selectors set the context. *Table 2.10* shows examples of combinators supported in D3.

Type	Example	Description
Descendant	`table td`	Selects elements that *descend* from the preceding selection.
Child	`table > tbody >sx tr > td`	Selects elements that are *children* of the preceding selection.
Next-sibling	`#sec2 + div + form`	Selects siblings that appear *after* the preceding selection.
Subsequent-sibling	`#sec2 ~ form`	Selects siblings that immediately *follow* the elements represented by the preceding selector.

Table 2.10 – W3C combinators (for contextual selections). The result is the last selector in the expression

Examples can be found in the `Selectors/` folder.

This section covered essential methods and properties used in dynamic HTML pages. SVG-specific DOM methods are covered in *Chapter 3*. See the *References* section for links to specs and documentation.

The Canvas API

Canvas is an HTML element that provides a graphical context for interactive graphics. Like SVG, it allows you to draw shapes, lines, and text with JavaScript. It's used in data visualization libraries, tools, games, animations, and interactive applications.

Most of your D3 applications will render SVG, but shape generators in D3 can also generate Canvas, which you may decide to use for performance if you have too many objects in memory. You may also choose to use Canvas for other reasons, such as to integrate your visualization with a tool that requires Canvas.

This section provides a brief overview of the Canvas API, listing the methods you are most likely to use and providing an example using different shapes, styles, and transitions.

How to create a Canvas

To draw using Canvas, you need to create a graphics context using the <canvas> element somewhere in the <body> of your page. This can be done with plain HTML:

```
<body><canvas id="canvas" width="400" height="300"></canvas></body>
```

You can also generate a canvas programmatically using any API that manipulates the document, such as the DOM. Of course, you can also use D3, as shown here:

```
d3.select("body").append("canvas").attr("width", 400).attr("height", 300);
```

The preceding code selects the parent node (<body>) where the canvas will be placed, appends the <canvas> to it, and sets its `height` and `width` attributes.

If you declare the <canvas> element in HTML, you can reference it by its ID using the DOM or D3. Using D3, you must convert its reference to a standard DOM node to use the Canvas API:

```
const canvas = d3.select("#canvas").node(); // node() returns the DOM object
```

Once you have a canvas object, you can obtain its 2D graphics context and start drawing:

```
const ctx = canvas.getContext("2d");
```

Most of the API consists of methods and properties that are called from the graphics context object. Before starting to draw, you need to set its properties, such as `font`, `fillStyle`, and `strokeStyle`:

```
ctx.fillStyle = "red";
ctx.strokeStyle = "rgba(255,127,0,0.7)";
ctx.lineWidth = 10;
```

Then you can *fill* or *stroke* shapes. The following will draw a 50x50 pixel red square with a 10-pixel-wide semi-transparent yellow border at 50,50:

```
ctx.fillRect(50,50,50,50);
ctx.strokeRect(50,50,50,50);
```

You can also draw other shapes, text, and images on the same canvas. The context's properties won't change unless they are redefined or a saved context is restored. It's a good practice to save the context to the stack before applying properties or transforms and restore it when you are done drawing an object:

```
ctx.save();
 ctx.transform(50,60);
 ctx.scale(2);     /* … drawing commands … */
ctx.restore();
```

Some essential Canvas commands are listed in the following sections.

Graphics context

Before running any operation on the canvas, such as drawing a line or a string of text, you first need to set the desired properties in the graphics context. *Table 2.11* lists properties that modify the state of the graphics context and allow you to store and retrieve it when necessary:

Property or method	Description
`fillStyle`	Sets the color to be used in `fill()` commands.
`strokeStyle`	Sets the color to be used in `stroke()` commands.
`lineWidth`	Sets the line width to be used in `stroke()` commands.
`lineCap`	Sets the style of the line caps: `'butt'` (default), `'round'`, or `'square'`.
`textAlign`	Sets text alignment: `'start'` (default), `'center'`, `'end'`, `'left'`, or `'right'`.
`textBaseline`	Sets text baseline: `'middle'`, `'hanging'`, `'top'`, `'bottom'`, and so on.
`font`	Sets the font for text commands, using the compact CSS font syntax.
`globalAlpha`	Sets the global opacity (`0` = transparent, `1` = opaque) for the context.
`shadowBlur`, `shadowColor`, `shadowOffsetX`, `shadowOffsetY`	Sets shadow properties. The default color is transparent black. The default numeric values are zero.
`setLineDash(`*dasharray*`)`	Sets the dash array (alternating line and space lengths) for strokes.
`translate(`x,y`)`	Sets the current `translate` transform for the context.
`scale(`x,y`)`	Sets the current `scale` transform for the context.
`rotate(`*angle*`)`	Sets the current `rotate` transform for the context. Angle is in radians.
`save()`	Saves the state of the current context (pushes into a stack).
`restore()`	Restores the state of the last context saved (pops it from the stack).

Table 2.11 – Methods and properties that modify the state of the Canvas context

After setting up the graphics context, you will use methods to start drawing in it.

Drawing rectangles, text, and images

Table 2.12 lists context methods used to draw rectangles, text, and images:

Method	Description
`fillRect(`x,y,w,h`);`	Fills a rectangle. Used to clear the canvas on redrawing.
`strokeRect(`x,y,w,h`)`	Draws a border around a rectangle.

Method	Description
`fillText(`*text*`,`*x*`,`*y*`);`	Fills text at *x*, *y* (depends on current `textAlign` and `textBaseline`).
`strokeText(`*text*`,`*x*`,`*y*`);`	Draws a border around text.
`drawImage(`*image*`,`*x*`,`*y*`,`*w*`,`*h*`)`	Draws an image at *x*, *y* with width *w* and height *h*.

Table 2.12 – Canvas context methods used to draw rectangles, text, and images

The `fillRect()` command is typically used to clear the canvas or part of it before redrawing, but you can also use it to draw rectangles. To draw any other shape, you need to create a path.

Drawing arbitrary shapes

A **path** is a series of commands to draw lines, curves, or arcs. It starts with `ctx.beginPath()`, followed by path commands (*Table 2.13*), then `fill()` and/or `stroke()` to draw it with the current styles.

Method	Description
`beginPath()`	Starts a path.
`closePath()`	Closes a path.
`moveTo(`*x*`,`*y*`)`	Moves the cursor to a position in the path.
`lineTo(`*x*`,`*y*`)`	Draws a line to a position in the path.
`bezierCurveTo(`*c1x*`,`*c1y*`,`*c2x*`,`*c2y*`,`*x*`,`*y*`)` `quadraticCurveTo(`*cx*`,`*cy*`,`*x*`,`*y*`)`	Draws curves with one (quadratic) or two (Bezier) control points.
`arc(`*x*`,`*y*`,`*r*`,`*sa*`,`*ea*`)`	Draws an arc by specifying the center, radius, start, and end angles.
`arcTo(`*sx*`,`*sy*`,`*r*`,`*ex*`,`*ey*`)`	Draws an arc by specifying the coordinates of the starting point, radius, and coordinates of the endpoint.
`rect(`*x*`,`*y*`,`*w*`,`*h*`)`	Draws a rectangle in a path with top-left corner coordinates, width, and height.
`clip()`	Creates a clipping region with the shapes drawn by the path that will affect objects that are drawn afterwards.
`fill()`	Fills a path with the current color.
`stroke()`	Strokes the path with the current color.

Table 2.13 – Canvas methods used to render paths

You can create identical graphics using SVG or Canvas, but the way they work is very different. SVG makes it easy to access and manipulate individual shapes using DOM commands, while Canvas renders graphics as a bitmap. See `Canvas/1-canvas-svg-compare.html`, which attempts to generate the same image using SVG and Canvas. SVG is covered in detail in *Chapter 3*.

Data formats

Data used in visualizations is usually distributed or delivered in a standard text format that can be shared. Popular proprietary formats such as Excel spreadsheets are common, but most statistical data, especially in public data portals, is delivered in CSV, XML, or JSON formats.

CSV

CSV, or **Comma-Separated Values**, is a very popular tabular text format containing one header row with column names, and one or more data rows with value fields. Rows are separated by line breaks, and the comma-separated fields form columns. A simple CSV with the population and land area of seven continents (`Chapter02/data/continents.csv`) looks like this:

```
continent,population,areakm2
"North America",579024000,24490000
"Asia",4436224000,43820000
"Europe",738849000,10180000
"Africa",1216130000,30370000
"South America",422535000,17840000
"Oceania",39901000,9008500
"Antarctica",1106,13720000
```

JavaScript doesn't come with a native CSV parser, but a very small and simple CSV file can be parsed using string manipulation tools or regular expressions. Larger files that may contain commas inside strings, missing data, and so on require a CSV parser. In this book, we will use the one provided by D3 (`d3.csv()`).

JSON

JSON stands for **JavaScript Object Notation**. It looks a lot like a JavaScript object, with stricter formation rules. It's probably the easiest format to work with. It's compact and easy to parse. The continent data in JSON format (`Chapter02/data/continents.json`) looks like this:

```
[
    {"continent": "North America", "population": 579024000, "areakm2": 24490000},
    {"continent": "Asia", "population": 4436224000, "areakm2": 43820000},
    {"continent": "Europe", "population": 738849000, "areakm2": 10180000},
    {"continent": "Africa", "population": 1216130000, "areakm2": 30370000},
    {"continent": "South America", "population": 422535000,"areakm2": 17840000},
    {"continent": "Oceania","population": 39901000, "areakm2": 9008500},
    {"continent": "Antarctica", population": 1106, "areakm2": 13720000}
]
```

JSON is the preferred format for data manipulation in JavaScript. To convert it into an object and access its properties with the *dot* operator, you can use `JSON.parse()`:

```
const obj = JSON.parse(jsonString);
```

Once you parse a JSON file, you can navigate its properties using array and object operations:

```
const popEurope = obj[2].population;
```

Sometimes you need to convert a JavaScript object back into JSON format. You might also do this for debugging. This can be done with `JSON.stringify()`:

```
const jsonString = JSON.stringify(obj);
```

If you are using D3, you don't need to use these native DOM functions, since it's much simpler and easier to use `d3.json()`, which loads and parses JSON in one step.

XML

XML – eXtensible Markup Language – is still a popular data format. It has native support in JavaScript via DOM APIs and doesn't require parsing. Although, you may still find data in XML, CSV, and JSON is more popular and usually smaller and easier to work with. The XML file with the same data as the CSV and JSON files is much larger (`Chapter02/data/continents.xml`):

```
<continents>
    <continent>
        <name>North America</name>
        <population>579024000</population>
        <area unit="km">24490000</area>
    </continent>
    <continent>
        <name>Asia</name>
        <population>4436224000</population>
        <area unit="km">43820000</area>
    </continent>
    <!-- … +4 <continent/> blocks were omitted … -->
    <continent>
        <name>Antarctica</name>
        <population>1106</population>
        <area unit="km">13720000</area>
    </continent>
</continents>
```

XML files can be validated with an XML schema. You can extract data from a well-formed XML file with DOM or XPath. There are tools in all languages to manipulate XML, which is also very easy to generate. Its main disadvantage is its verbosity and size. You can load and parse XML in D3 using `d3.xml()`.

You will find many examples that parse, search, and extract data from CSV, JSON, and XML files, using standard and third-party tools in pure JavaScript, in the `DataFormat/` folder.

Displaying a map with JSON, JavaScript, and Canvas

Let's finish this chapter by drawing a world map using Canvas and pure JavaScript. The data is in a standard JSON format that stores shapes for countries in geographical coordinates (`data/world-lowres.geojson`). The file's standard structure is shown in the following fragment:

```
{"type":"FeatureCollection","features":[
    {"type":"Feature","id":"AFG","properties":{"name":"Afghanistan"},
     "geometry":{"type":"Polygon","coordinates":[[[61.210817,35.650072],...]]
    },
    {"type":"Feature", "id":"AGO", "properties":{"name":"Angola"},
     "geometry":{"type":"MultiPolygon","coordinates":[[[[16.326528,-5.87747,...]]
    },
    /* ... +many other lines ... */ ]
}
```

We will load and parse the data, get each longitude and latitude, scale the values to fit in the canvas, and then draw each shape with path commands. The Canvas code is in a module, imported by the page, which parses the file to get the `features` object: an array with all the countries (`Canvas/2-canvas-map.html`):

```
<canvas id="map" width="1000" height="500"></canvas>
<script type="module">
    import {drawMap} from './js/canvasmap.js';
    fetch('data/world.geojson').then(response => response.text())
        .then(jsonData => drawMap(JSON.parse(jsonData).features));
</script>
```

The preceding code imports the `canvasmap.js` module, containing the code that will draw the shapes. It sets up the canvas, styles and strokes, and exports the `drawMap()` function called from the page:

```
const canvas = document.getElementById("map");
const ctx = canvas.getContext("2d");

ctx.fillStyle = 'white';          // map ocean background
ctx.fillRect(0, 0, canvas.width, canvas.height);
ctx.lineWidth = .25;
ctx.strokeStyle = 'white';
ctx.fillStyle = 'rgb(50,100,150)';     // settings for country borders and background

export function drawMap(data) {
    data.forEach(obj => {
       if(obj.geometry.type === 'MultiPolygon') {
           obj.geometry.coordinates.forEach(poly => drawPolygon(poly[0]));
       } else if(obj.geometry.type === 'Polygon') {
           obj.geometry.coordinates.forEach(poly => drawPolygon(poly));
       }
    });
}
```

Coordinate pairs are latitudes and longitudes, which need to be scaled to fit in the canvas. A pair of scaling functions computes the lines rendered by the canvas in `drawPolygon()`, which draws each country:

```
const scaleX = coord => canvas.width * (180 + coord) / 360;
const scaleY = coord => canvas.height * (90 - coord) / 180;

function drawPolygon(coords) {
    ctx.beginPath();
    for(let i = 0; i < coords.length; i++ ) {
       let latitude = coords[i][1];
       let longitude = coords[i][0];
        if(i == 0) {
            ctx.moveTo(scaleX(longitude), scaleY(latitude));
        } else {
            ctx.lineTo(scaleX(longitude), scaleY(latitude));
        }
    }
    ctx.stroke();    // draw country borders
    ctx.fill();      // fill country background
}
```

The result is a map of the world painted in an HTML page using Canvas (*Figure 2.6*).

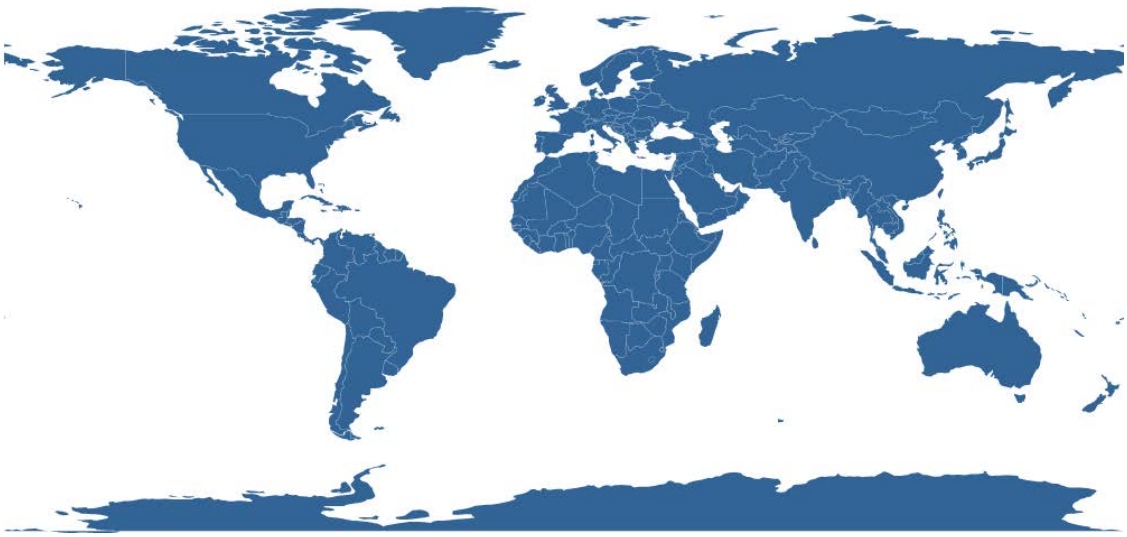

Figure 2.5 – A world map created using GeoJSON, JavaScript, and Canvas. Code: Canvas/2-canvas-map.html

As you can see, standard web technologies such as JavaScript and Canvas can work like magic to render very cool things. In this book, you will learn how to create this map using even less code.

Summary

This chapter provided a review of important topics in web technologies you should know about to get the most out of D3. More than half the chapter was dedicated to reviewing JavaScript concepts, such as fundamental JavaScript data structures (arrays, objects, strings, maps, and sets), standard ECMAScript modules, and asynchronous functions. Other important topics that were explored include the Document Object Model, the Canvas API, and popular data formats, such as CSV and JSON. The last section described a complete example using a bit of everything that was covered.

In the next chapter, we will cover the essential SVG concepts you should know about before using D3.

References

1. *ECMA JavaScript specification* (ECMAScript 2019): 262.ecma-international.org/10.0/
2. *MDN JavaScript reference*: developer.mozilla.org/en-US/docs/Web/JavaScript
3. *W3C DOM Level 3*: www.w3.org/TR/DOM-Level-3-Core/
4. *MDN DOM reference*: developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
5. *W3C Selectors Specification*: www.w3.org/TR/selectors-api/
6. *MDN Canvas reference*: developer.mozilla.org/en-US/docs/Web/API/Canvas_API/
7. *WhatWG Canvas specification*: html.spec.whatwg.org/multipage/canvas.html
8. *GeoJSON Specification* (RFC 7946): datatracker.ietf.org/doc/html/rfc7946/
9. The data file `data/world-lowres.geojson` was generated from the *Natural Earth* public domain dataset: naturalearthdata.com/, which contains public domain shapefiles for GIS applications. The shapefiles were converted to GeoJSON using mapshaper.org.